· base_report_to_printer · Full Documentation

Module: Report to Printer (OCA)

Version: 18.0.1.2.0
License: AGPL-3
Author: Agile Business Group, Pegueroles SCP, NaN, LasLabs, Camptocamp, OCA
Category: Generic Modules / Base
Depends: `web`
External Dependency: `pycups` (Python library)

· Table of Contents

---

1. Overview

The **base_report_to_printer** module connects Odoo 18 to **CUPS (Common UNIX Printing System)** servers, allowing reports to be sent **directly to physical printers** attached to the server instead of downloading PDF files to the browser.

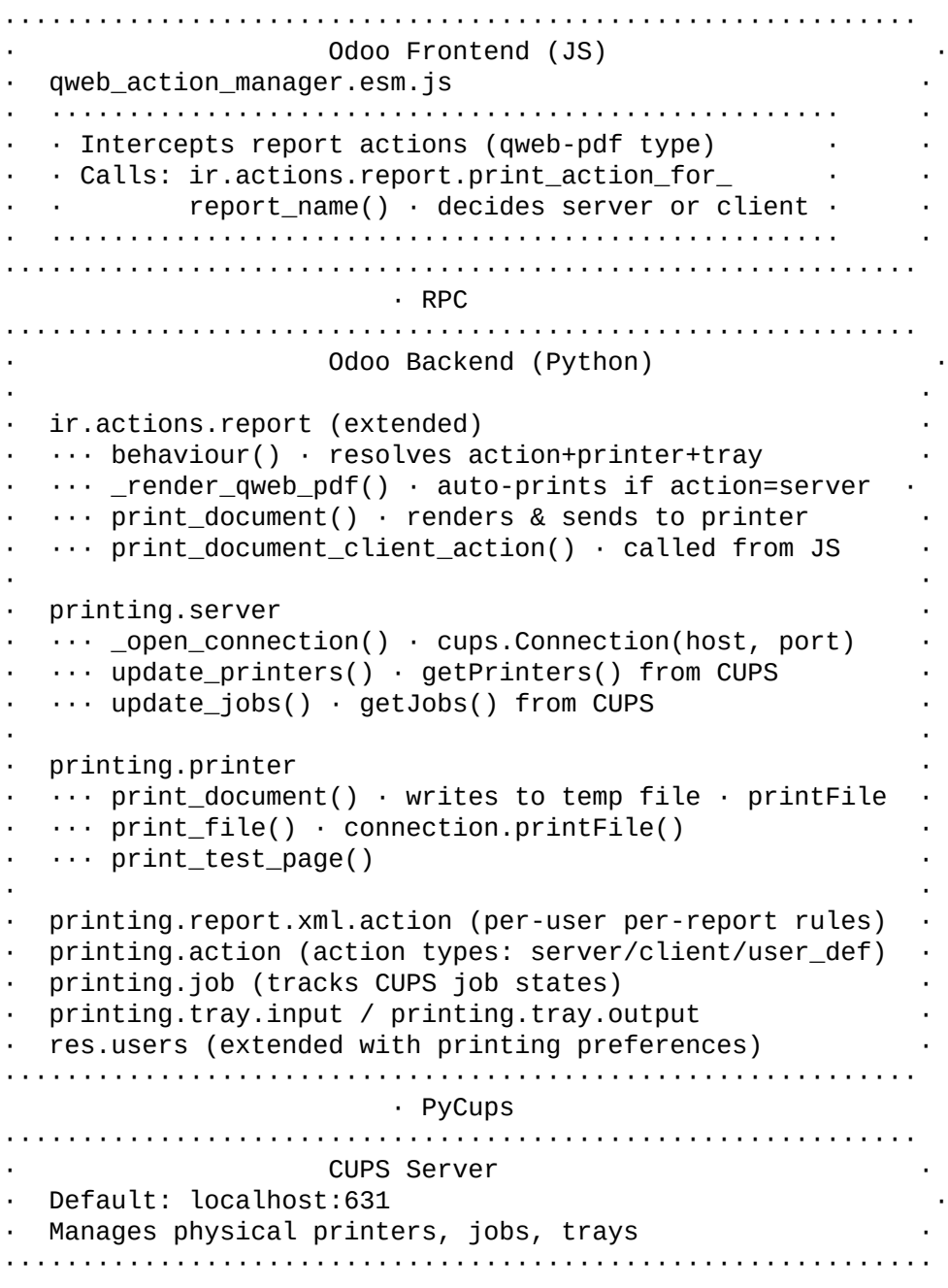
What it enables:

- **Direct server-side printing** · click "Print" and the document goes straight to the printer
- **PDF download fallback** · if the printer is unavailable, the user can still download the PDF
- **Per-user, per-report, and global configuration** · full flexibility
- **Paper tray / output bin selection** · route documents to specific trays (e.g., preprinted letterhead)
- **Multi-threaded printing** · non-blocking print jobs

- **Print job tracking** · monitor job status (pending, processing, completed, etc.)
- **Print attachments** · send PDF attachments directly to a printer

---

2. Architecture & Models



---

3. Prerequisites

3.1 Install CUPS on the Server

CUPS must be installed on the **same server as Odoo** (or a reachable network CUPS server).

```
# Ubuntu / Debian
sudo apt-get update
sudo apt-get install cups
sudo apt-get install libcups2-dev
sudo apt-get install python3-dev
# Start & enable CUPS service
sudo systemctl enable cups
sudo systemctl start cups
```

3.2 Install PyCups Python Library

```
# Inside your Odoo virtual environment
pip install pycups
```

·**Important:** `pycups` only works on Linux/macOS. It does NOT work natively on Windows. If your Odoo server runs on Linux (which is typical for production), this is fine. For odoo.sh or Docker, ensure `libcups2-dev` is installed.

3.3 Add Printers to CUPS

Access the CUPS web interface at `http://your-server:631` and add your printers there first. Odoo will then discover them.

```
# Verify CUPS is running
lpstat -t
# List available printers
lpstat -p -d
```

---

4. Installation in Odoo 18

1. Place the `base_report_to_printer` folder in your **custom addons path**
2. Update the addons list: **Settings · Apps · Update Apps List**
3. Search for **"Report to printer"** and click **Install**
4. The module will automatically create:
   - Two default **Printing Actions**: "Send to Printer" and "Send to Client"
   - A **cron job** that updates printer jobs every minute
   - Default behaviour set to **"Send to Client"** (PDF download) for all reports

---

5. Step-by-Step Configuration

5.1 Add a CUPS Print Server

Menu: `Settings · Printing · Servers`

Field	Description	Default
Name	Friendly name for the

server | "Localhost" | | **Address** | IP address or hostname of the CUPS server | "localhost" | | **Port** | CUPS port number | 631 | | **User** | Username for CUPS authentication (optional) | Empty | | **Password** | Password for CUPS authentication (optional) | Empty | | **Encryption Policy** | HTTP encryption mode (optional) | Empty | | **Multi Thread** | Enable non-blocking threaded printing | False |

Encryption Policy Options:

- `HTTP_ENCRYPT_IF_REQUESTED` · encrypt only if requested
- `HTTP_ENCRYPT_NEVER` · never encrypt
- `HTTP_ENCRYPT_REQUIRED` · encryption required
- `HTTP_ENCRYPT_ALWAYS` · always encrypt

Example · Local Server:

```
Name:    Local CUPS
Address: localhost
Port:    631
```

Example · Remote Server:

```
Name:    Office Print Server
Address: 192.168.1.100
Port:    631
User:    admin
Password: ****
```

5.2 Discover & Update Printers

After adding a server, you need to discover the printers:

Option A · From the Server form:

1. Open the server record
2. Click **"Update Printers"** button in the header
3. Odoo connects to CUPS and fetches all available printers

Option B · From the menu:

1. Go to `Settings · Printing · Update Printers from CUPS`
2. A wizard opens · Click **"OK"**
3. You'll see a list of all discovered printers

Each printer record will show: | Field | Description |
|---|---| | **Name** | Printer display name (from CUPS `printer-info`) | | **System Name** | Internal CUPS name (used for API calls) | | **Server** | The parent CUPS server | | **URI** | Device URI (e.g., `ipp://...`, `usb://...`) | | **Model** | Make and model from PPD | | **Location** | Physical location (set in CUPS) | | **Status** | `available`, `printing`, `error`, `unavailable`, etc. | | **Paper Sources** | Input trays detected from PPD | | **Output Trays** | Output bins detected from PPD | | **Multi Thread** | Inherited from server, can be overridden |

Printer Actions Available:

- **Enable** · enable a disabled printer
- **Disable** · disable a printer in CUPS
- **Cancel All Running Jobs** · cancel all active jobs on this printer

- **Print Test Page** · send a test page to verify connectivity
- **Set Default / Unset Default** · mark a printer as the system default

5.3 Set a Default Printer

1. Go to `Settings · Printing · Printers`
2. Open a printer record
3. Click "**Set Default**"

> Only ONE printer can be the default at any time. Setting a new default automatically unsets the previous one.

The default printer is used as a **fallback** when no specific printer is configured for a report or user.

5.4 Configure Report Behaviour (Global)

Menu: `Settings · Printing · Reports`
(This is a shortcut to the standard **Settings · Technical · Actions · Reports**)

For each report, a new "**Print**" tab is added with these fields:

Field	Description
Default Behaviour	`Send to Printer`, `Send to Client`, or `Use user's defaults`
Default Printer	Specific printer for this report (overrides system default)
Paper Source	Input tray on the selected printer
Output Bin	Output tray on the selected printer
Specific actions per user	Per-user overrides (see 5.6)

Action Types Explained:

Action Type	What Happens
Send to Printer	Report is rendered server-side and sent directly to the CUPS printer
Send to Client	Report is rendered as PDF and downloaded to the user's browser (normal Odoo)
Use user's defaults	Falls back to the printing action configured in the user's preferences

Example · Auto-print Sales Orders:

1. Find report `sale.report_saleorder`
2. Go to the **Print** tab
3. Set **Default Behaviour** = `Send to Printer`
4. Set **Default Printer** = your desired printer
5. Optionally select a **Paper Source** (e.g., Tray 2 for letterhead)

5.5 Configure Per-User Defaults

Menu: `Settings · Users & Companies · Users · [Select User] · Preferences`

Under the "**Printing**" section:

Field	Description
Printing Action	`Send to Printer` or `Send to Client`
Default Printer	User's preferred printer

> Users can also configure this themselves from their own **My Profile / Preferences** page.

Example · User "John" always prints directly:

1. Open John's user form
2. Under Preferences · Printing:
   - **Printing Action** = `Send to Printer`
   - **Default Printer** = `HP LaserJet Office`

5.6 Configure Per-User Per-Report Rules

This is the most granular level of configuration. You can set different printers and actions for each **user + report** combination.

Location: On the Report form · **Print** tab · **Specific actions per user** section

Field	Description
User	The specific user
Action	`Send to Printer`, `Send to Client`, or `Use user's defaults`
Printer	Specific printer for this user+report combo
Paper Source	Input tray
Output Bin	Output tray

Example · Different printers for different users on Invoice report:

User	Action	Printer	Paper Source
John	Send to Printer	HP LaserJet Office	Tray 2
Sarah	Send to Printer	Epson Warehouse	Auto
Admin	Send to Client	·	·

5.7 Select Paper Source / Output Tray

Paper sources (input trays) and output bins are **automatically detected** from the printer's PPD (PostScript Printer Description) file when printers are updated from CUPS.

Where you can set trays:

1. **On the Report form** · Print tab · Paper Source / Output Bin
2. **On per-user per-report rules** · Paper Source / Output Bin
3. **On the User preferences** · Default Printer Paper Source / Output Bin

   Use case: You have preprinted letterhead in Tray 2 and want invoices to always print from Tray 2.

---

6. How Direct Printing Works (Full Flow)

Here's the complete step-by-step flow when a user clicks "Print" on a report:

Step 1: User Clicks Print

The user clicks a print button (e.g., "Print Invoice") in the Odoo interface.

Step 2: JavaScript Intercepts the Action

The `qweb_action_manager.esm.js` file registers a handler in the `ir.actions.report handlers` registry. When a `qweb-pdf` report action is triggered:

```
· It calls
ir.actions.report.print_action_for_report_name(report_name)
```

```
· This returns: { action: "server"|"client", printer_name: "...",
printer_exception: true|false }
```

Step 3: Decision Point

If action = "server" AND no printer exception:

- The JS calls `ir.actions.report.print_document_client_action(report_id, active_ids, data)`
- A success notification is shown: "Successfully sent to printer!"
- The PDF is NOT downloaded to the browser

If action = "server" AND printer exception (error/unavailable):

- A warning notification is shown with the printer name
- A "Print" button is offered that falls back to PDF download
- The user can still get their document

If action = "client":

- Normal Odoo behaviour · PDF is generated and downloaded to the browser

Step 4: Server-Side Rendering & Printing

When `print_document_client_action()` is called:

1. The report is rendered using `_render_qweb_pdf()` (with `must_skip_send_to_printer=True` to prevent double-printing)
2. The `behaviour()` method resolves the final action, printer, and tray settings
3. `printer.print_document()` is called:
    - Content is written to a **temporary file** on disk
    - `printer.print_file()` opens a CUPS connection
    - `connection.printFile(system_name, file_name, title, options)` sends the job to CUPS
    - The temporary file is deleted
4. CUPS handles the actual printing to the physical device

Step 5: Auto-Print on _render_qweb_pdf (Alternative Path)

Even without the JS handler, the `_render_qweb_pdf()` and `_render_qweb_text()` methods are overridden. If `behaviour()['action'] == 'server'` and the context doesn't have `must_skip_send_to_printer`, the report is **automatically** sent to the printer after rendering.

This means:

- Reports generated via **automated actions**, **cron jobs**, or **API calls** will also auto-print if configured with `action=server`

---

7. Priority / Behaviour Resolution Order

The `behaviour()` method resolves settings in this order (later overrides earlier):

```
1. User Default (lowest priority)
```

```
   ··· User's printing_action field ("Send to Printer" or "Send to
Client")
   ··· User's printing_printer_id (default printer)
   ··· User's printer_input_tray_id / printer_output_tray_id
   ··· Fallback: system default printer (printing.printer with
default=True)
2. Report Default (overrides user defaults)
   ··· Report's property_printing_action_id (if not "Use user's
defaults")
   ··· Report's printing_printer_id
   ··· Report's printer_input_tray_id / printer_output_tray_id
3. Per-User Per-Report Rule (highest priority)
   ··· printing.report.xml.action matching (report_id + user_id)
   ··· Only applied if action != "user_default"
   ··· Only non-empty values override (False values don't override)
```

Example Resolution:

Setting Source	Action	Printer	Tray
User "John" defaults	Send to Client	HP Office	Auto
Report "Invoice" defaults	Send to Printer	·	Tray 2
Per-User Per-Report (John+Inv)	·	Epson Warehouse	·
Final Result	**Send to Printer**	**Epson Warehouse**	**Tray 2**

Logic:

- Action: Report says "Send to Printer" · overrides user's "Send to Client"
- Printer: Per-user-per-report says "Epson Warehouse" · overrides user's "HP Office"
- Tray: Per-user-per-report has no tray · keeps Report's "Tray 2"

---

8. Multi-Threaded Printing

When **Multi Thread** is enabled (on server or printer level):

- The print job runs in a **separate Python thread** after the current database transaction commits
- The user gets an **immediate response** without waiting for the printer
- This is useful for slow printers or remote CUPS servers

How it works:

```
# In print_document_client_action():
if printer.multi_thread:
    @self.env.cr.postcommit.add
    def _launch_print_thread():
        threaded_calculation = threading.Thread(
            target=self.print_document_threaded, args=(...)
        )
        threaded_calculation.start()
```

·· **Caveat:** With threaded printing, there's no download fallback if the printing fails silently. The error won't reach the user's browser.

To enable:

1. Go to `Settings · Printing · Servers · [Your Server]`
2. Check **"Multi Thread"**
3. This propagates to all printers on that server (can be overridden per printer)

---

9. Print Jobs Management

Viewing Jobs

Jobs are visible on each **Printer** form under the **Jobs** tab.

Each job record contains: | Field | Description | |---|---| | Job Name | Name of the document that was printed | | Job ID (CUPS) | The CUPS-assigned job ID | | Media Progress | Percentage complete (progress bar) | | State | pending, processing, completed, canceled, aborted | | State Reason | Detailed reason (e.g., "job-printing", "cups-filter-crashed") | | Creation Date | When the job was submitted | | Processing Date | When printing started | | Completion Date | When the job finished |

Updating Jobs

- **Manually:** Open a server · click **"Update Jobs"**
- **Automatically:** A cron job runs **every 1 minute** and updates all job statuses

Canceling Jobs

- **Single job:** Open the job · click **"Cancel"**
- **All jobs on a printer:** Open the printer · click **"Cancel All Running Jobs"**

---

10. Print Attachments Wizard

The module includes a wizard to print **PDF attachments** directly.

Model: `wizard.print.attachment`

| Field | Description | |---|---| | Printer | Select the target printer | | Attachments to Print | List of attachment lines with copy counts |

Each line specifies:

- **Attachment** · must be PDF or octet-stream
- **Copies** · number of copies to print

The wizard decodes the base64 attachment data and sends it directly to the selected printer.

---

11. JavaScript Frontend Handler

File: `static/src/js/qweb_action_manager.esm.js`

This is registered as a `qweb-pdf` report handler in the Odoo web client:

```
registry.category("ir.actions.report handlers")
    .add("cups_report_action_handler", cupsReportActionHandler);
```

Flow:

1. **Check print action** · calls `print_action_for_report_name` RPC
2. **If action=server & no exception** · calls `print_document_client_action` RPC · shows success toast
3. **If action=server & exception** · shows warning notification with "Print" fallback button
4. **If action=client** · does nothing (falls through to default Odoo PDF download handler)

Context Keys Used:

Key	Purpose
`force_print_to_client`	Forces PDF download even if server printing is set
`must_skip_send_to_printer`	Prevents auto-print in `_render_qweb_pdf()`
`skip_printer_exception`	Ignores printer errors when checking behaviour

---

12. Security Groups & Access Rights

Groups:

Group	XML ID	Description
Printing / Print User	`printing_group_user`	Can view printers and jobs
Printing / Print Manager	`printing_group_manager`	Full CRUD on all printing objects

Automatic Group Assignment:

- **All internal users** (`base.group_user`) automatically get **Print User**
- **ERP Managers** (`base.group_erp_manager`) automatically get **Print Manager**

Access Rights Summary:

Model	Print User	Print Manager
printing.server	Read only	Full CRUD
printing.printer	Read only	Full CRUD
printing.job	Read only	Full CRUD
printing.action	Read only	Full CRUD
printing.report.xml.action	Read only	Full CRUD
printing.tray.input	Read only	Full CRUD
printing.tray.output	Read only	Full CRUD
printing.printer.update.wizard	·	Full CRUD
wizard.print.attachment	Full CRUD	Full CRUD
wizard.print.attachment.line	Full CRUD	Full CRUD

---

13. Automated Cron Job

Name: `Update Printers Jobs`
Frequency: Every 1 minute
Action: Calls `printing.server.action_update_jobs()` on all servers

This keeps the job status in Odoo synchronized with CUPS. It fetches:

- Job name, state, state reason
- Media progress percentage
- Creation, processing, and completion timestamps

Jobs that no longer exist in CUPS are marked as **inactive** (archived).

---

14. Troubleshooting

· "Cannot `import cups`"

- **Cause:** `pycups` is not installed
- **Fix:** `pip install pycups` in your Odoo venv. On Ubuntu, also install `libcups2-dev`

· "Failed to connect to the CUPS server on localhost:631"

- **Cause:** CUPS service is not running or not reachable
- **Fix:**

```
sudo systemctl start cups
sudo systemctl status cups
```

· Printer shows status "server-error"

- **Cause:** Odoo could not reach the CUPS server during the last update
- **Fix:** Check network connectivity, CUPS server status, and credentials

· Printer shows status "unavailable"

- **Cause:** The printer was not found in CUPS during the last update
- **Fix:** Check if the printer still exists in CUPS (`lpstat -p`)

· Reports still download as PDF instead of printing

- **Cause:** The behaviour is set to "Send to Client"
- **Fix:** Check these in order:
  1. User preferences · Printing Action = "Send to Printer"
  2. Report · Print tab · Default Behaviour = "Send to Printer"
  3. Report · Print tab · Default Printer is set
  4. The printer status is "available" (not "error" or "unavailable")

· "No printer configured to print this report"

- **Cause:** No printer resolved through the behaviour chain
- **Fix:** Set a printer on the report, user, or set a system default printer

· Print job sent but nothing comes out

- **Check:** `Settings · Printing · Printers · [Printer]` for job status
- **Check:** CUPS web interface at `http://localhost:631/jobs`
- **Check:** Physical printer connectivity

· Your Error: `KeyError: 'o'` in `sale.document_tax_totals`

- **Note:** This specific error is **NOT related** to `base_report_to_printer`. It's a QWeb template error in a customized `sale.document_tax_totals` template where the variable `o` (the sale order record) is missing from the rendering context. This is a report template issue, not a printing issue. Check if Web Studio or a custom module modified the `sale.document_tax_totals` template and used `o.gross_total` in a context where `o` is not available.

---

15. Complete Model Reference

`printing.server`

Represents a CUPS print server connection.

Field	Type	Description
name	Char	Server name
address	Char	IP/hostname
port	Integer	Port (default: 631)
user	Char	Auth username
password	Char	Auth password
encryption_policy	Selection	HTTP encryption mode
active	Boolean	Is server active
printer_ids	One2many	Related printers
multi_thread	Boolean	Enable threaded printing

Key Methods:

- `_open_connection(raise_on_error)` · creates `cups.Connection`
- `update_printers(domain, raise_on_error)` · syncs printers from CUPS
- `update_jobs(which, first_job_id)` · syncs jobs from CUPS

`printing.printer`

Represents a physical printer discovered from CUPS.

Field	Type	Description
name	Char	Display name
system_name	Char	CUPS internal name
server_id	Many2one	Parent server
default	Boolean	Is system default
status	Selection	available/printing/error/etc.
status_message	Char	CUPS status message
model	Char	Printer make & model
location	Char	Physical location
uri	Char	Device URI
input_tray_ids	One2many	Paper source trays
output_tray_ids	One2many	Output bins
multi_thread	Boolean	Threaded printing
job_ids	One2many	Print jobs

Key Methods:

- `print_document(report, content, **opts)` · print content (bytes/string)
- `print_file(file_name, report, **opts)` · print a file from disk
- `print_test_page()` · send test page to CUPS
- `set_default()` / `unset_default()` · manage default printer
- `enable()` / `disable()` · enable/disable in CUPS
- `cancel_all_jobs(purge_jobs)` · cancel all jobs

`printing.action`

Defines the available print action types.

| Action Type | Description | |---|---| | `server` | Send to Printer (direct CUPS printing) | | `client` | Send to Client (PDF download) | | `user_default` | Use the user's default printing action |

Default data created on install:

- "Send to Printer" (`action_type=server`)
- "Send to Client" (`action_type=client`)

`printing.report.xml.action`

Per-user per-report printing rules (the most granular configuration).

| Field | Type | Description | |---|---|---| | report_id | Many2one | The report | | user_id | Many2one | The user | | action | Selection | server/client/user_default | | printer_id | Many2one | Specific printer | | printer_input_tray_id | Many2one | Paper source | | printer_output_tray_id | Many2one | Output bin |

`printing.job`

Tracks print jobs synced from CUPS.

| Field | Type | Description | |---|---|---| | name | Char | Job name | | job_id_cups | Integer | CUPS job ID | | printer_id | Many2one | Printer used | | server_id | Many2one | Server (related) | | job_media_progress | Integer | Progress % | | job_state | Selection | pending/processing/completed/etc. | | job_state_reason | Selection | Detailed reason | | time_at_creation | Datetime | When submitted | | time_at_processing | Datetime | When started printing | | time_at_completed | Datetime | When finished |

`printing.tray.input` / `printing.tray.output`

Paper source trays and output bins detected from printer PPD.

| Field | Type | Description | |---|---|---| | name | Char | Display name | | system_name | Char | CUPS/PPD internal name | | printer_id | Many2one | Parent printer |

`res.users` (extended)

User preferences for printing.

| Field | Type | Description | |---|---|---| | printing_action | Selection | server or client | | printing_printer_id | Many2one | User's default printer | | printer_input_tray_id | Many2one | Default paper source | | printer_output_tray_id | Many2one | Default output bin |

`ir.actions.report` (extended)

Report definitions extended with printing settings.

Field	Type	Description
property_printing_action_id	Many2one	Default behaviour (company-dependent)
printing_printer_id	Many2one	Default printer for this report
printer_input_tray_id	Many2one	Paper source
printer_output_tray_id	Many2one	Output bin
printing_action_ids	One2many	Per-user rules

Quick Setup Checklist

- [ ] CUPS installed and running on server
- [ ] `pycups` Python library installed
- [ ] Printers added in CUPS web interface
- [ ] Module installed in Odoo
- [ ] Server record created in `Settings · Printing · Servers`
- [ ] Printers updated from CUPS (click "Update Printers")
- [ ] Default printer set (optional)
- [ ] User preferences configured (`Preferences · Printing`)
- [ ] Report behaviour set to "Send to Printer" on desired reports
- [ ] Printer selected on desired reports
- [ ] Test print successful

Documentation generated from source code analysis of base_report_to_printer v18.0.1.2.0